

Galileo — Scroll-revm & Stateless Block Verifier

— Security Assessment

Authors

- Rohit Narurkar
- Huaihuaqing Zhang
- Date: 20 November 2025

Executive Summary

Severity	Count	Identifiers	Potential Impact
Informational	8	I-1 → I-8	All issues are rated <i>Informational</i> and concern unused features, non-idiomatic code, and avoiding panics; no higher-severity issues were reported.

Scope

- **Stateless Block Verifier** — <https://github.com/scroll-tech/stateless-block-verifier/tree/40aed436537900b49338b37325c4444e3b408ad8>
- **Scroll REVM** — <https://github.com/scroll-tech/scroll-revm/tree/a1ac004adf0019d9926defc4e31e6a76a7e558f7>

1 · Finding Overview

ID	Finding	Severity
I-1	Features <code>cycle-tracker</code> and <code>sp1</code> are never used	Informational
I-2	<code>SparseState::clear_storage</code> is not the most idiomatic	Informational
I-3	Possible panic on <code>unwrap</code> in <code>SparseState::storage</code>	Informational
I-4	Undesirable <code>unwraps</code> in <code>SparseState::calculate_state_root</code>	Informational
I-5	Hard-coded copied static gas cost	Informational
I-6	<code>make_scroll_instruction_table</code> could be <code>const fn</code>	Informational
I-7	Redundant condition in <code>skip_balance_check</code>	Informational
I-8	EIP-3607 check could reuse upstream <code>revm</code> code	Informational

2 · Detailed Findings

2.1 Unused features `cycle-tracker` and `sp1` in `sbv-trie` (I-1)

Severity: Informational

Component: `stateless-block-verifier` (`Cargo.toml`, crate `sbv-trie`)

Description The features `cycle-tracker` and `sp1` are declared for the crate `sbv-trie` but are never used. Keeping unused features in `Cargo.toml` adds noise and can confuse future maintainers about the actual feature set of the crate.

Recommendation Remove the unused features `cycle-tracker` and `sp1` from `Cargo.toml` if they are not intended to be used.

2.2 `SparseState::clear_storage` is not the most idiomatic (I-2)

Severity: Informational

Component: `stateless-block-verifier` (`src/lib.rs`)

Description The `clear_storage` method in `SparseState` uses hashbrown's `Entry` API to obtain the entry for an address in the `storages` map and, if the entry is vacant (`Entry::Vacant`), inserts the default value.

Since only the default value of `RlpTrie` is ever inserted, there is a more idiomatic way to express this logic using `entry(...).or_default()` directly.

Recommendation Refactor `clear_storage` to use the more idiomatic pattern:

```
/// Clears the storage of an account.
fn clear_storage(&mut self, hashed_address: B256) -> &mut RlpTrie<U256> {
    self.storages
        .get_mut()
        .entry(hashed_address)
        .or_default()
}
```

This keeps the existing behaviour while simplifying the code.

2.3 Possible panic on unwrap in `SparseState::storage` (I-3)

Severity: Informational

Component: `stateless-block-verifier` (`src/lib.rs`)

Description The implementation of `SparseState::storage` is:

```

/// Returns the storage slot value that corresponds to the given (address, slot) tuple.
fn storage(&self, address: Address, slot: U256) -> Result<U256, ProviderError> {
    let storages = self.storages.borrow();
    // storage() is always be called after account(), so the storage trie must already exist
    let storage_trie = storages.get(&keccak256(address)).unwrap();
    Ok(storage_trie
        .get(keccak256(B256::from(slot)))?
        .unwrap_or(U256::ZERO))
}

```

This code uses `unwrap()` on the result of `storages.get(&keccak256(address))` and relies on the comment’s assumption that `storage()` is *always* called after `account()`, so that the storage trie must exist.

The issue is that `SparseState` itself does not control how it is used. For example, it may be used by `WitnessDatabase`, `revm`, or future forks / upstream versions where the invariant “`account()` is always called before `storage()`” no longer holds. In such cases, `unwrap()` could panic at runtime.

Recommendation Since `SparseState::storage` already returns a `Result`, a missing storage trie should be handled explicitly as an error instead of panicking.

```

ProviderError::TrieWitnessError("missing storage for
account {address}")

```

2.4 Undesirable unwraps in `SparseState::calculate_state_root` (I-4)

Severity: Informational

Component: `stateless-block-verifier` (`src/lib.rs`)

Description `SparseState::calculate_state_root` contains multiple `unwrap()` calls, in particular on the result returned from `SparseState::storage_trie_mut`. The internal behaviour depends on `risc-ethereum`’s implementation; relying on `unwrap()` here can lead to panics if assumptions change or unexpected errors occur in the underlying code.

Recommendation Avoid panicking on fallible operations in `calculate_state_root`. Instead, handle errors from `SparseState::storage_trie_mut` and propagate them using the existing `StatelessValidationError::Custom` variant.

2.5 Hard-coded copied static gas cost (I-5)

Severity: Informational

Component: `scroll-revm` (`src/instructions.rs`)

Description When overriding instructions, static gas costs have been copied from upstream into the override logic. Since the gas costs themselves were not changed, duplicating them locally increases maintenance overhead: any upstream change in gas costs would have to be manually mirrored.

Recommendation Reuse the static gas cost from the original instruction table instead of hard-coding copies. The report suggests the following pattern:

```
macro_rules! override_instruction {
    ($opcode:expr, $new:ident) => {
        table[$opcode as usize] = Instruction::new(
            $new:<WIRE, HOST>,
            table[$opcode as usize].static_gas(),
        );
    };
}

override_instruction!(opcode::BLOCKHASH, blockhash);
override_instruction!(opcode::BASEFEE, basefee);
override_instruction!(opcode::TSTORE, tstore);
override_instruction!(opcode::TLOAD, tload);
override_instruction!(opcode::SELFDESTRUCT, selfdestruct);
override_instruction!(opcode::MCOPY, mcopy);
override_instruction!(opcode::DIFFICULTY, difficulty);
override_instruction!(opcode::CLZ, clz);
```

This keeps static gas costs in one place (upstream) and reduces the risk of divergence.

2.6 make_scroll_instruction_table could be const fn (I-6)

Severity: Informational

Component: scroll-revm (src/instructions.rs)

Description The function `make_scroll_instruction_table` can be written as a `const fn`. Doing so would allow the instruction table to be constructed at compile time, potentially improving startup behaviour and aligning better with upstream patterns.

Recommendation Change the function signature to be `const fn` as suggested:

```
pub const fn make_scroll_instruction_table<WIRE: InterpreterTypes, HOST: ScrollContextTr>(<
```

Ensure that the body of the function remains compatible with `const fn` requirements.

2.7 Redundant condition in skip_balance_check (I-7)

Severity: Informational

Component: scroll-revm (src/handler.rs)

Description The skip_balance_check flag is computed as:

```
let skip_balance_check = tx.is_l1_msg() && spec.is_enabled_in(ScrollSpecId::EUCLID);
```

This expression is used inside an outer if where tx.is_l1_msg() is already known to be true. In that context, the additional tx.is_l1_msg() check is redundant.

Recommendation Simplify the condition to rely solely on the spec check:

```
let skip_balance_check = spec.is_enabled_in(ScrollSpecId::EUCLID);
```

This preserves the behaviour in the current context while removing the redundant condition.

2.8 EIP-3607 check could reuse upstream revm code (I-8)

Severity: Informational

Component: scroll-revm (src/handler.rs)

Description The EIP-3607 check for L1 messages is currently implemented by copying logic that already exists in upstream revm. Duplicating this logic can lead to subtle drift if upstream behaviour changes, and it increases maintenance cost.

Recommendation Reuse the upstream helper pre_execution::validate_account_nonce_and_code from revm instead of maintaining a separate copy. The report suggests calling it as follows:

```
pre_execution::validate_account_nonce_and_code(  
    &mut caller_account.info,  
    tx.nonce(),  
    is_eip3607_disabled,  
    true,  
)?;
```

This centralises EIP-3607 behaviour in the upstream helper and reduces the risk of divergence.